JAVA COLLECTIONS FRAMEWORK

Practical Programs — Combined Report 2

A compiled report of 11 Java examples covering ArrayDeque, ArrayList, HashMap, HashSet, Hashtable, LinkedHashMap, LinkedHashSet, LinkedList, TreeMap, PriorityQueue and Stack.

Sections for every program
Aim / Objective
System Requirements
Algorithm
UML-style class relationship
Source Code
Compilation and Execution Commands
Actual Program Output
Result

Output panels are formatted terminal-style text, not screenshots captured from the user's computer.

Contents

1. ArrayDequeExample
2. ArrayListExample
3. HashMapExample
4. HashSetExample
5. HashtableExample
6. LinkedHashMapExample
7. LinkedHashSetExample
8. LinkedListExample
9. TreeMapExample
10. PriorityQueueExample
11. StackExample

Program 1: ArrayDequeExample

Aim / Objective

Demonstrate insertion and removal at both ends of an ArrayDeque.

System Requirements

JDK 8 or later; a text editor or Java IDE; command-line terminal.

Algorithm

1. Create an ArrayDeque of passenger names.
2. Add three passengers.
3. Insert a VIP passenger at the front.
4. Remove one passenger from the front and one from the rear.
5. Display the remaining passengers.

UML / Class Relationship (simplified)

```
+----------------------------+
| ArrayDequeExample |
+----------------------------+
| main(String[] args)        |
+----------------------------+
            | uses
            v
ArrayDeque; add(), addFirst(), removeFirst(), removeLast().
```

Source Code

```
import java.util.ArrayDeque;

public class ArrayDequeExample {
    public static void main(String[] args) {
        ArrayDeque<String> passengers = new ArrayDeque<>();
        // Adding passengers
        passengers.add("Ravi");
        passengers.add("Anu");
        passengers.add("Kiran");
        System.out.println("Passengers: " + passengers);
        // Add VIP passenger at front
        passengers.addFirst("VIP");
        System.out.println("After adding VIP: " + passengers);
        // Remove passenger from front
        System.out.println("Boarded Passenger: "
                + passengers.removeFirst());
        // Remove passenger from rear
        System.out.println("Removed from Rear: "
                + passengers.removeLast());
        System.out.println("Remaining Passengers: " + passengers);
    }
}
```

Compilation and Execution Commands

```
javac ArrayDequeExample.java
java ArrayDequeExample
```

Output (actual run)

```
Passengers: [Ravi, Anu, Kiran]
After adding VIP: [VIP, Ravi, Anu, Kiran]
Boarded Passenger: VIP
Removed from Rear: Kiran
Remaining Passengers: [Ravi, Anu]
```

Result

Compiled and executed successfully during report generation. ArrayDeque; add(), addFirst(), removeFirst(), removeLast().

Program 2: ArrayListExample

Aim / Objective

Demonstrate adding, accessing, updating, removing, and counting list elements.

System Requirements

JDK 8 or later; a text editor or Java IDE; command-line terminal.

Algorithm

1. Create an ArrayList of shopping items.
2. Add four items and display the list.
3. Display the first item.
4. Replace Bread with Biscuits.
5. Remove Milk and display the final list and size.

UML / Class Relationship (simplified)

```
+----------------------------+
| ArrayListExample |
+----------------------------+
| main(String[] args)        |
+----------------------------+
            | uses
            v
ArrayList; add(), get(), set(), remove(), size().
```

Source Code

```
import java.util.ArrayList;

public class ArrayListExample {
    public static void main(String[] args) {
        ArrayList<String> items = new ArrayList<>();
        // Adding items
        items.add("Rice");
        items.add("Milk");
        items.add("Bread");
        items.add("Eggs");
        System.out.println("Shopping List: " + items);
        // Accessing an item
        System.out.println("First Item: " + items.get(0));
        // Updating an item
        items.set(2, "Biscuits");
        System.out.println("Updated List: " + items);
        // Removing an item
        items.remove("Milk");
        System.out.println("After Removal: " + items);
        // Size
        System.out.println("Total Items: " + items.size());
    }
}
```

Compilation and Execution Commands

```
javac ArrayListExample.java
java ArrayListExample
```

Output (actual run)

```
Shopping List: [Rice, Milk, Bread, Eggs]
First Item: Rice
Updated List: [Rice, Milk, Biscuits, Eggs]
After Removal: [Rice, Biscuits, Eggs]
Total Items: 3
```

Result

Compiled and executed successfully during report generation. ArrayList; add(), get(), set(), remove(), size().

Program 3: HashMapExample

Aim / Objective

Demonstrate key-value storage, lookup, key checking, and removal with HashMap.

System Requirements

JDK 8 or later; a text editor or Java IDE; command-line terminal.

Algorithm

1. Create a HashMap for product names and prices.
2. Insert four products.
3. Print the map and retrieve the price of Bag.
4. Check whether Pen is present.
5. Remove Bottle and display the map.

UML / Class Relationship (simplified)

```
+----------------------------+
| HashMapExample |
+----------------------------+
| main(String[] args)        |
+----------------------------+
            | uses
            v
HashMap; put(), get(), containsKey(), remove(). HashMap does not guarantee iteration order.
```

Source Code

```
import java.util.HashMap;

public class HashMapExample {
    public static void main(String[] args) {
        HashMap<String, Integer> products = new HashMap<>();
        // Adding products and prices
        products.put("Notebook", 50);
        products.put("Pen", 20);
        products.put("Bag", 800);
        products.put("Bottle", 300);
        System.out.println("Products: " + products);
        // Getting price
        System.out.println("Price of Bag: "
                + products.get("Bag"));
        // Checking product
        System.out.println("Is Pen available? "
                + products.containsKey("Pen"));
        // Removing product
        products.remove("Bottle");
        System.out.println("After Removal: " + products);
    }
}
```

Compilation and Execution Commands

```
javac HashMapExample.java
java HashMapExample
```

Output (actual run)

```
Products: {Notebook=50, Bottle=300, Pen=20, Bag=800}
Price of Bag: 800
Is Pen available? true
After Removal: {Notebook=50, Pen=20, Bag=800}
```

Result

Compiled and executed successfully during report generation. HashMap; put(), get(), containsKey(), remove(). HashMap does not guarantee iteration order.

Program 4: HashSetExample

Aim / Objective

Demonstrate unique set elements, membership testing, removal, and size.

System Requirements

JDK 8 or later; a text editor or Java IDE; command-line terminal.

Algorithm

1. Create a HashSet of platform numbers.
2. Add values including a duplicate.
3. Display the unique values.
4. Check whether platform 5 exists.
5. Remove platform 3 and display the set and size.

UML / Class Relationship (simplified)

```
+----------------------------+
| HashSetExample |
+----------------------------+
| main(String[] args)        |
+----------------------------+
            | uses
            v
HashSet; add(), contains(), remove(), size(). HashSet does not guarantee iteration order.
```

Source Code

```
import java.util.HashSet;

public class HashSetExample {
    public static void main(String[] args) {
        HashSet<Integer> platforms = new HashSet<>();
        platforms.add(2);
        platforms.add(5);
        platforms.add(1);
        platforms.add(3);
        platforms.add(2);
        System.out.println("Platform Numbers: " + platforms);
        // Checking a platform
        System.out.println("Is Platform 5 available? "
                + platforms.contains(5));
        // Removing a platform
        platforms.remove(3);
        System.out.println("After Removal: " + platforms);
        System.out.println("Total Platforms: "
                + platforms.size());
    }
}
```

Compilation and Execution Commands

```
javac HashSetExample.java
java HashSetExample
```

Output (actual run)

```
Platform Numbers: [1, 2, 3, 5]
Is Platform 5 available? true
After Removal: [1, 2, 5]
Total Platforms: 3
```

Result

Compiled and executed successfully during report generation. HashSet; add(), contains(), remove(), size(). HashSet does not guarantee iteration order.

Program 5: HashtableExample

Aim / Objective

Demonstrate key-value operations with Hashtable for emergency contacts.

System Requirements

JDK 8 or later; a text editor or Java IDE; command-line terminal.

Algorithm

1. Create a Hashtable for emergency contact names and numbers.
2. Insert four contacts.
3. Display the table and retrieve the ambulance number.
4. Check whether Police is present.
5. Remove Fire Service and display the updated table.

UML / Class Relationship (simplified)

```
+----------------------------+
| HashtableExample |
+----------------------------+
| main(String[] args)        |
+----------------------------+
            | uses
            v
Hashtable; put(), get(), containsKey(), remove(). Hashtable does not guarantee iteration order and disallows nu
```

Source Code

```
import java.util.Hashtable;

public class HashtableExample {
    public static void main(String[] args) {
        Hashtable<String, String> contacts = new Hashtable<>();
        // Adding contacts
        contacts.put("Ambulance", "108");
        contacts.put("Police", "100");
        contacts.put("Fire Service", "101");
        contacts.put("Women Helpline", "181");
        System.out.println("Emergency Contacts: " + contacts);
        // Getting phone number
        System.out.println("Ambulance Number: "
                + contacts.get("Ambulance"));
        // Checking a contact
        System.out.println("Is Police Number Available? "
                + contacts.containsKey("Police"));
        // Removing a contact
        contacts.remove("Fire Service");
        System.out.println("After Removal: " + contacts);
    }
}
```

Compilation and Execution Commands

```
javac HashtableExample.java
java HashtableExample
```

Output (actual run)

```
Emergency Contacts: {Ambulance=108, Police=100, Women Helpline=181, Fire Service=101}
Ambulance Number: 108
Is Police Number Available? true
After Removal: {Ambulance=108, Police=100, Women Helpline=181}
```

Result

Compiled and executed successfully during report generation. Hashtable; put(), get(), containsKey(), remove(). Hashtable does not guarantee iteration order and disallows null keys and values.

Program 6: LinkedHashMapExample

Aim / Objective

Demonstrate a map that preserves insertion order while storing restaurant ratings.

System Requirements

JDK 8 or later; a text editor or Java IDE; command-line terminal.

Algorithm

1. Create a LinkedHashMap of restaurant names and ratings.
2. Insert four restaurants.
3. Display ratings and retrieve Tasty Hub's rating.
4. Remove Food Corner.
5. Display the remaining entries in insertion order.

UML / Class Relationship (simplified)

```
+----------------------------+
| LinkedHashMapExample |
+----------------------------+
| main(String[] args)        |
+----------------------------+
            | uses
            v
LinkedHashMap; put(), get(), remove(). Iteration follows insertion order by default.
```

Source Code

```
import java.util.LinkedHashMap;

public class LinkedHashMapExample {
    public static void main(String[] args) {
        LinkedHashMap<String, Double> restaurants = new LinkedHashMap<>();
        restaurants.put("Spice Garden", 4.5);
        restaurants.put("Food Corner", 4.2);
        restaurants.put("Tasty Hub", 4.7);
        restaurants.put("Urban Bites", 4.3);
        System.out.println("Restaurant Ratings:");
        System.out.println(restaurants);
        // Getting rating
        System.out.println("Rating of Tasty Hub: "
                + restaurants.get("Tasty Hub"));
        // Removing an entry
        restaurants.remove("Food Corner");
        System.out.println("After Removal:");
        System.out.println(restaurants);
    }
}
```

Compilation and Execution Commands

```
javac LinkedHashMapExample.java
java LinkedHashMapExample
```

Output (actual run)

```
Restaurant Ratings:
{Spice Garden=4.5, Food Corner=4.2, Tasty Hub=4.7, Urban Bites=4.3}
Rating of Tasty Hub: 4.7
After Removal:
{Spice Garden=4.5, Tasty Hub=4.7, Urban Bites=4.3}
```

Result

Compiled and executed successfully during report generation. LinkedHashMap; put(), get(), remove(). Iteration follows insertion order by default.

Program 7: LinkedHashSetExample

Aim / Objective

Demonstrate a set that removes duplicates while preserving insertion order.

System Requirements

JDK 8 or later; a text editor or Java IDE; command-line terminal.

Algorithm

1. Create a LinkedHashSet of daily tasks.
2. Add four tasks and repeat one task.
3. Display the unique tasks.
4. Check whether Practice Coding exists.
5. Remove Read Notes and display the set.

UML / Class Relationship (simplified)

```
+----------------------------+
| LinkedHashSetExample |
+----------------------------+
| main(String[] args)        |
+----------------------------+
            | uses
            v
LinkedHashSet; add(), contains(), remove().
```

Source Code

```
import java.util.LinkedHashSet;

public class LinkedHashSetExample {
    public static void main(String[] args) {
        LinkedHashSet<String> tasks = new LinkedHashSet<>();
        tasks.add("Study Java");
        tasks.add("Complete Assignment");
        tasks.add("Practice Coding");
        tasks.add("Read Notes");
        tasks.add("Study Java");
        System.out.println("Daily Tasks: " + tasks);
        // Checking a task
        System.out.println("Is Coding Practice Available? "
                + tasks.contains("Practice Coding"));
        // Removing a task
        tasks.remove("Read Notes");
        System.out.println("After Removal: " + tasks);
    }
}
```

Compilation and Execution Commands

```
javac LinkedHashSetExample.java
java LinkedHashSetExample
```

Output (actual run)

```
Daily Tasks: [Study Java, Complete Assignment, Practice Coding, Read Notes]
Is Coding Practice Available? true
After Removal: [Study Java, Complete Assignment, Practice Coding]
```

Result

Compiled and executed successfully during report generation. LinkedHashSet; add(), contains(), remove().

Program 8: LinkedListExample

Aim / Objective

Demonstrate adding, removing, and accessing elements in a LinkedList.

System Requirements

JDK 8 or later; a text editor or Java IDE; command-line terminal.

Algorithm

1. Create a LinkedList of bus stops.
2. Add four stops and display them.
3. Add Airport at the beginning and Hospital at the end.
4. Remove Market.
5. Display the updated list and first stop.

UML / Class Relationship (simplified)

```
+----------------------------+
| LinkedListExample |
+----------------------------+
| main(String[] args)        |
+----------------------------+
            | uses
            v
LinkedList; add(), addFirst(), addLast(), remove(), getFirst().
```

Source Code

```
import java.util.LinkedList;

public class LinkedListExample {
    public static void main(String[] args) {
        LinkedList<String> stops = new LinkedList<>();
        stops.add("Railway Station");
        stops.add("Market");
        stops.add("College");
        stops.add("Bus Stand");
        System.out.println("Bus Stops: " + stops);
        // Add a stop at the beginning
        stops.addFirst("Airport");
        // Add a stop at the end
        stops.addLast("Hospital");
        System.out.println("Updated Bus Stops: " + stops);
        // Remove a stop
        stops.remove("Market");
        System.out.println("After Removal: " + stops);
        // Display first stop
        System.out.println("First Stop: " + stops.getFirst());
    }
}
```

Compilation and Execution Commands

```
javac LinkedListExample.java
java LinkedListExample
```

Output (actual run)

```
Bus Stops: [Railway Station, Market, College, Bus Stand]
Updated Bus Stops: [Airport, Railway Station, Market, College, Bus Stand, Hospital]
After Removal: [Airport, Railway Station, College, Bus Stand, Hospital]
First Stop: Airport
```

Result

Compiled and executed successfully during report generation. LinkedList; add(), addFirst(), addLast(), remove(), getFirst().

Program 9: TreeMapExample

Aim / Objective

Demonstrate sorted key-value entries with employee IDs as keys.

System Requirements

JDK 8 or later; a text editor or Java IDE; command-line terminal.

Algorithm

1. Create a TreeMap of employee IDs and departments.
2. Insert four employee entries.
3. Display the sorted map.
4. Retrieve the department for employee 103.
5. Display the first and last employee IDs.

UML / Class Relationship (simplified)

```
+----------------------------+
| TreeMapExample |
+----------------------------+
| main(String[] args)        |
+----------------------------+
            | uses
            v
TreeMap; put(), get(), firstKey(), lastKey(). Keys are sorted naturally.
```

Source Code

```
import java.util.TreeMap;

public class TreeMapExample {
    public static void main(String[] args) {
        TreeMap<Integer, String> employees = new TreeMap<>();
        employees.put(104, "HR");
        employees.put(101, "CSE");
        employees.put(103, "Finance");
        employees.put(102, "Marketing");
        System.out.println("Employee Details: " + employees);
        // Get department of employee
        System.out.println("Department of Employee 103: "
                + employees.get(103));
        // First key
        System.out.println("First Employee ID: "
                + employees.firstKey());
        // Last key
        System.out.println("Last Employee ID: "
                + employees.lastKey());
    }
}
```

Compilation and Execution Commands

```
javac TreeMapExample.java
java TreeMapExample
```

Output (actual run)

```
Employee Details: {101=CSE, 102=Marketing, 103=Finance, 104=HR}
Department of Employee 103: Finance
First Employee ID: 101
Last Employee ID: 104
```

Result

Compiled and executed successfully during report generation. TreeMap; put(), get(), firstKey(), lastKey(). Keys are sorted naturally.

Program 10: PriorityQueueExample

Aim / Objective

Demonstrate priority-based processing with a PriorityQueue of integers.

System Requirements

JDK 8 or later; a text editor or Java IDE; command-line terminal.

Algorithm

1. Create a PriorityQueue of ticket numbers.
2. Add four priorities.
3. Display the queue and inspect its head with peek().
4. Remove the head using poll().
5. Display the remaining queue.

UML / Class Relationship (simplified)

```
+----------------------------+
| PriorityQueueExample |
+----------------------------+
| main(String[] args)        |
+----------------------------+
            | uses
            v
PriorityQueue; add(), peek(), poll(). The head is the smallest integer; printed queue order is not guaranteed t
```

Source Code

```
import java.util.PriorityQueue;

public class PriorityQueueExample {
    public static void main(String[] args) {
        PriorityQueue<Integer> tickets = new PriorityQueue<>();
        tickets.add(40);
        tickets.add(10);
        tickets.add(30);
        tickets.add(20);
        System.out.println("Ticket Priorities: " + tickets);
        // Highest priority = smallest number
        System.out.println("Next Ticket: " + tickets.peek());
        // Removing the highest priority ticket
        System.out.println("Completed Ticket: " + tickets.poll());
        System.out.println("Remaining Tickets: " + tickets);
    }
}
```

Compilation and Execution Commands

```
javac PriorityQueueExample.java
java PriorityQueueExample
```

Output (actual run)

```
Ticket Priorities: [10, 20, 30, 40]
Next Ticket: 10
Completed Ticket: 10
Remaining Tickets: [20, 40, 30]
```

Result

Compiled and executed successfully during report generation. PriorityQueue; add(), peek(), poll(). The head is the smallest integer; printed queue order is not guaranteed to be sorted.

Program 11: StackExample

Aim / Objective

Demonstrate stack operations for undoing actions using LIFO order.

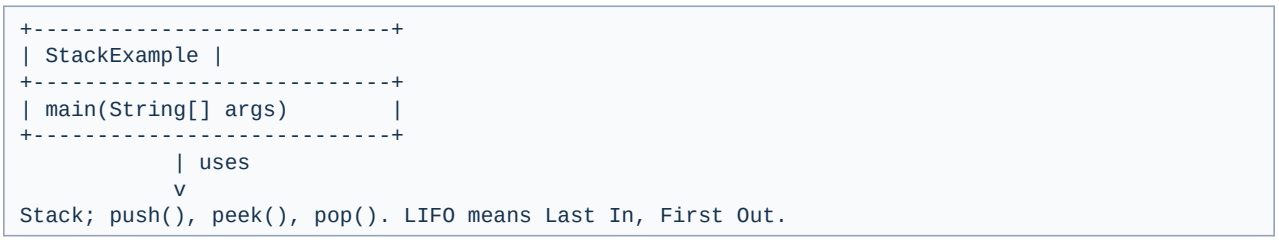
System Requirements

JDK 8 or later; a text editor or Java IDE; command-line terminal.

Algorithm

1. Create a Stack of editing actions.
2. Push four actions onto the stack.
3. Display the stack and peek at the top action.
4. Pop the last action to undo it.
5. Display the remaining actions.

UML / Class Relationship (simplified)



Source Code

```
import java.util.Stack;

public class StackExample {
    public static void main(String[] args) {
        Stack<String> actions = new Stack<>();
        actions.push("Type Text");
        actions.push("Add Image");
        actions.push("Change Font");
        actions.push("Save File");
        System.out.println("Actions: " + actions);
        // View top action
        System.out.println("Last Action: " + actions.peek());
        // Undo last action
        System.out.println("Undo Action: " + actions.pop());
        System.out.println("Remaining Actions: " + actions);
    }
}
```

Compilation and Execution Commands

```
javac StackExample.java
java StackExample
```

Output (actual run)

```
Actions: [Type Text, Add Image, Change Font, Save File]
Last Action: Save File
Undo Action: Save File
Remaining Actions: [Type Text, Add Image, Change Font]
```

Result

Compiled and executed successfully during report generation. Stack; push(), peek(), pop(). LIFO means Last In, First Out.